Índice general

Capítulo 5

Desarrollo e implementación del prototipo

ExploraNica es un sistema de gestión turística desarrollado con .NET 10, Windows Forms, ASP.NET Core, Entity Framework Core y PostgreSQL. Se implementaron dos aplicaciones de escritorio sobre una API común: el backoffice del operador y el panel administrativo. El sistema centraliza el registro de agencias, la gestión documental, el catálogo de tours, la disponibilidad de salidas y la administración de reservas.

La arquitectura distribuye las responsabilidades entre dominio, aplicación, infraestructura y presentación. Las reglas de reserva y validación se ejecutan en el servidor, mientras los clientes presentan formularios, tablas e indicadores para la operación diaria. Las integraciones de pago, reseñas y acceso móvil forman parte del plan de ampliación del producto.

5.1. Arquitectura y estructura

5.1.1. Diagrama de arquitectura

La solución comprende dos clientes de escritorio y una API central. El cliente del operador permite gestionar la actividad de la agencia; el cliente administrativo concentra la supervisión. Ambos intercambian datos con la API mediante HTTP y JSON. La API resuelve autenticación, autorización y operaciones del negocio, mientras PostgreSQL conserva los registros. Windows Forms proporciona los controles de interfaz utilizados por los clientes [**?**].

La organización distingue dominio, aplicación, infraestructura y presentación. El dominio define entidades y estados; aplicación contiene contratos, DTO y validadores; infraestructura implementa servicios, acceso a datos y almacenamiento documental; presentación contiene los controladores HTTP y los formularios de escritorio. La inyección de dependencias conecta las interfaces con sus implementaciones y permite mantener las responsabilidades de cada

proyecto.

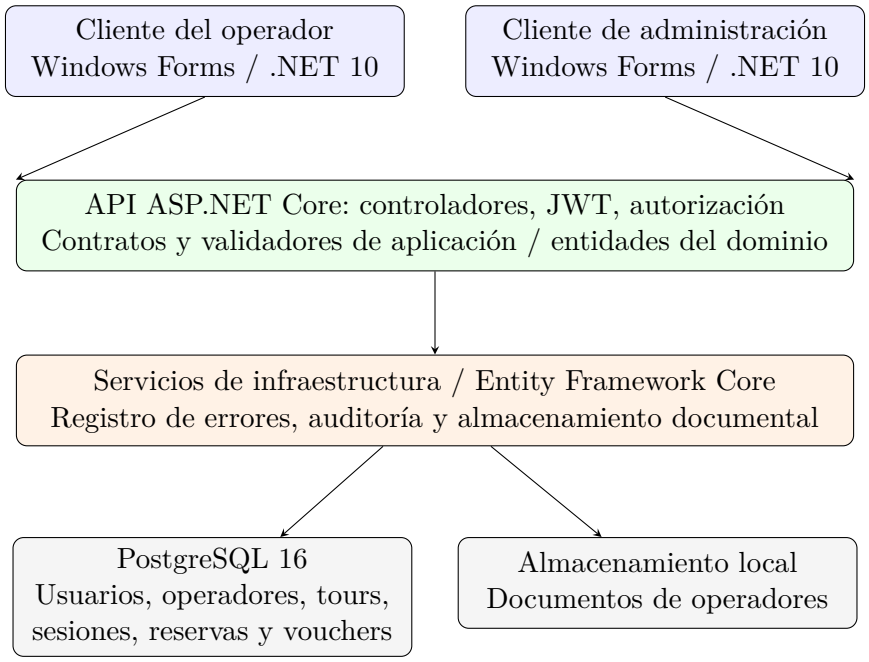


Figura 5.1: Arquitectura de ExploraNica y comunicación entre sus componentes.

JWT identifica al usuario y transporta los atributos de su sesión. La API valida el token y comprueba los permisos antes de ejecutar las operaciones protegidas [**?**]. Los clientes concentran la interacción con el usuario y consumen los servicios mediante solicitudes HTTP con cuerpos JSON.

5.1.2. Estructura de carpetas y archivos

La raíz contiene `ExploraNica.slnx`, `README.md`, `docker-compose.yml`, los scripts de preparación y el directorio de documentación `odd/tasks`. La estructura principal es la siguiente:

```
ExploraNica/
  ExploraNica.slnx
  README.md
  docker-compose.yml
  scripts/
  odd/tasks/
  src/
    Core/
      ExploraNica.Domain/
        Entities/  Enums/  Exceptions/  Builders/
      ExploraNica.Application/
        DTOs/  Interfaces/  Validators/
    Infrastructure/
      ExploraNica.Infrastructure/
        Services/  Persistence/  Middleware/
      ExploraNica.Shared/
        Configuration/  Services/  UI/
    Presentation/
      ExploraNica.Api/
        Controllers/  Middleware/  Health/  OpenApi/
      ExploraNica.OperatorApp/
        Forms/  Services/  Resources/
      ExploraNica.AdminApp/
        Forms/  Controls/  Resources/
```

`ExploraNicaDbContext.cs` coordina el acceso a datos y las migraciones conservan la evolución del esquema. `Program.cs` de la API registra servicios y configura el procesamiento de solicitudes. Los archivos `appsettings.json` contienen las opciones de ejecución, que pueden sobrescribirse con variables de entorno. En los clientes, `Api:BaseUrl` define la dirección del servidor.

5.2. Módulos implementados

5.2.1. Descripción de cada módulo: propósito, entradas y salidas

Tabla 5.1: Módulos de ExploraNica.

Módulo	Propósito	Entradas	Salidas
Autenticación	Comprobar identidad y permitir acceso según rol.	Correo y contraseña; configuración JWT.	Token firmado, vencimiento o error de credenciales.
Operadores y documentos	Registrar agencias y mantener su expediente.	Datos personales y comerciales; RUC, identificación y documento institucional.	Cuenta pendiente; documentos registrados; estado y motivo de rechazo.
Verificación administrativa	Revisar agencias y resolver su solicitud.	Operador seleccionado, documentos y decisión; motivo si se rechaza.	Estado aprobado o rechazado y actualización visible al operador.
Catálogo de tours	Crear y actualizar las ofertas de la agencia; supervisar su activación.	Título, descripción, destino, precio y duración.	Tour registrado o actualizado; indicador de actividad.
Sesiones y disponibilidad	Definir salidas y consultar los cupos.	Tour, fecha, hora, cupos y modificador de precio.	Sesión asociada al tour y disponibilidad consultable.
Reservas y vouchers	Registrar reservas y validar su utilización.	Sesión, cantidad de cupos e identidad autenticada; reserva a validar.	Reserva confirmada, cupo descontado; reserva completada y voucher usado.
Paneles y seguimiento	Mostrar información operativa y registrar acciones.	Listas de operadores, tours y reservas; eventos del sistema.	Conteos de control, errores correlacionados y registros de auditoría.

`AuthService` verifica las contraseñas con `IPasswordHasher<User>` y genera un token JWT con vencimiento. El sistema utiliza tres roles: `Admin`, `TourOperator` y `Tourist`. Los controladores restringen las operaciones por rol y los servicios resuelven la identidad autenticada para consultar o modificar los recursos asociados al usuario.

`OperatorRegistrationService`, `OperatorDocumentService` y `AdminService` implementan el registro, la carga documental y la verificación administrativa. Los estados son `Pending`, `Approved` y `Rejected`. Las pantallas `UcVendors`, `VendorDetailForm`, `VendorEditForm` y `VendorDocumentsForm` permiten consultar, editar y resolver los expedientes de las agencias.

`OperatorService` gestiona los tours de la agencia autenticada. `BookingService` calcula el importe de la reserva, descuenta los cupos disponibles y registra la confirmación dentro de una transacción. Durante la validación del servicio, actualiza o crea el voucher, registra su utilización y cambia la reserva a completada.

5.2.2. Interacción entre módulos

El inicio de sesión habilita las consultas del cliente y la API obtiene la identidad desde los atributos del token. La agencia se resuelve en el servidor para consultar sus tours y documentos. La creación de un tour desde el servicio del operador exige una cuenta aprobada; posteriormente se asocian sesiones y se consultan las reservas de cada salida.

La validación conecta reserva, sesión, tour y operador. Antes de completar la reserva, el servicio comprueba que el tour pertenece al operador autenticado. El cambio de estado, la actualización del voucher y el evento de auditoría se guardan dentro de una transacción. Los paneles administrativos consultan operadores, tours y reservas para presentar los indicadores de seguimiento.

5.3. Funcionalidades implementadas

5.3.1. Funcionalidades principales

Se implementaron inicio de sesión; registro de operadores por API con documentos; consulta, sustitución y reenvío de documentación; aprobación y rechazo administrativo; consulta y edición de agencias; creación y edición de tours; activación administrativa de ofertas; registro de sesiones; creación de reservas por API; consulta de reservas por sesión y validación de vouchers desde el cliente del operador.

El cliente administrativo incorpora listados de operadores, tours y reservas, además de un tablero con operadores pendientes y verificados, tours activos y reservas confirmadas. El backoffice del operador organiza sus opciones en panel, tours, documentos y reservas, con navegación desde un formulario principal.

5.3.2. Correspondencia entre funcionalidades y el proceso propuesto

Tabla 5.2: Correspondencia entre las funciones y el proceso de gestión.

Proceso	Funcionalidad	Resultado operativo
Alta y verificación	Registro de agencias, carga documental y decisión administrativa.	Expediente con estado y motivo de rechazo cuando corresponde.
Catálogo	Creación y edición de tours; activación administrativa de ofertas.	Oferta asociada a la agencia responsable.
Disponibilidad	Registro de sesiones con fecha, hora y cupos.	Salidas organizadas por tour.
Reserva	Creación transaccional y actualización condicionada de disponibilidad.	Reserva confirmada y cupos descontados.
Validación y cierre	Comprobación de propiedad, estado de la reserva y uso del voucher.	Reserva completada y utilización registrada.
Seguimiento	Conteos de operadores, tours y reservas; eventos de auditoría.	Indicadores de operación y trazabilidad de acciones.

Las funcionalidades siguen una secuencia operativa: registrar y verificar la agencia, mantener el catálogo, programar salidas, registrar reservas y confirmar la prestación del servicio. La API relaciona estas actividades mediante identificadores de operador, tour, sesión y reserva. Los controles de propiedad y estado conservan la continuidad del proceso entre los dos clientes de escritorio.

5.4. Validaciones implementadas

5.4.1. Validaciones de datos de entrada

Los validadores de aplicación comprueban que las reservas indiquen una sesión y una cantidad de cupos mayor que cero. Para los tours exigen título y ubicación, ambos de hasta

250 caracteres, precio base no negativo y duración positiva. El validador de sesiones exige tour, cupos positivos y una fecha posterior a la hora UTC de referencia con tolerancia de un minuto.

En el registro de operadores se normalizan los textos y se revisan campos obligatorios, formato de correo, longitudes compatibles con el modelo, contraseña de al menos ocho caracteres y unicidad de correo y RUC. El expediente requiere tres tipos documentales: RUC, identidad y constitución institucional. Los archivos se limitan a 5 MiB, se admiten PDF, PNG y JPEG y se verifica su firma binaria, además del tipo declarado. La sustitución documental no se permite si la cuenta ya está aprobada.

La edición administrativa limita el nombre de agencia a 200 caracteres, el RUC a 50 y el teléfono a 50 cuando se proporciona. Un rechazo requiere motivo. Para validar un voucher, la reserva debe pertenecer al operador, estar confirmada y no haber sido utilizada. En las descargas se filtra por propietario y se devuelven nombres controlados, evitando exponer rutas internas.

5.4.2. Manejo de errores y mensajes de usuario

`ErrorHandlingMiddleware` transforma las excepciones de dominio en respuestas JSON con código, mensaje e identificador de correlación. Los errores de validación de dominio incluyen un mapa por campo. La validación automática de modelos se procesa mediante ASP.NET Core. El cliente utiliza estas respuestas para comunicar el problema y orientar la corrección de los datos.

Tabla 5.3: Códigos y mensajes de error del sistema.

HTTP	Situación	Mensaje o respuesta representativa
400	Regla de negocio	“No hay cupos suficientes para la cantidad solicitada.”
401	Credenciales inválidas	“Credenciales inválidas.”
403	Operación prohibida	“Tu cuenta debe estar aprobada para publicar tours.”
404	Recurso inexistente	“No se encontró el documento solicitado.”
409	Conflicto	“El RUC ya está registrado por otro operador.”
422	Validación de dominio	“El motivo de rechazo es obligatorio.” y errores por campo.

HTTP	Situación	Mensaje o respuesta representativa
500	Fallo no previsto	“Ocurrió un error inesperado.” con correlación.

Los clientes incorporan `UiErrorHandler` y utilidades para actualizar controles desde el hilo de interfaz. Estos componentes centralizan los mensajes de error de las operaciones asíncronas y permiten recuperar el control de la pantalla después de una excepción. La API relaciona los registros técnicos con la respuesta mediante el identificador de correlación y limita los detalles de excepciones al entorno de desarrollo.

5.5. Estructuras de datos y lógica de programación

5.5.1. Estructuras de datos usadas

Las entidades principales son `User`, `Operator`, `OperatorDocument`, `Tour`, `TourSession`, `Booking`, `Voucher` y `AuditLog`. Los identificadores son de tipo `Guid`; los importes se calculan con valores decimales y las fechas de negocio emplean `DateTimeOffset`. Los enumerados `UserRole`, `BookingStatus` y `DocumentType` delimitan opciones conocidas.

Las listas y colecciones de DTO transportan resultados a las tablas de Windows Forms. Las consultas LINQ filtran y proyectan información, con `AsNoTracking` en varias consultas de lectura. Los diccionarios `Dictionary<string,string>` asocian un campo con su error; las listas de documentos preparados permiten verificar archivos antes de escribirlos. Las relaciones entre entidades se conservan con claves foráneas y configuraciones de Entity Framework Core.

La relación operador–tour y tour–sesión es de uno a muchos. La reserva referencia una sesión y un usuario. El voucher se relaciona con una reserva y registra el código, el estado de uso y la fecha de validación. Los documentos se vinculan a la agencia y se distinguen por su tipo, tamaño, formato y fecha de carga.

5.5.2. Lógica y algoritmos principales

Creación de reserva. El servicio abre una transacción dentro de la estrategia de reintentos de EF Core. Ejecuta una actualización condicionada que descuenta cupos únicamente cuando la sesión conserva disponibilidad suficiente. Si ninguna fila cambia, informa falta de cupo. Después carga la sesión y su tour, calcula el total, crea una reserva confirmada y confirma la transacción. Ante un fallo previo a la confirmación, el descuento no debe persistir independientemente de la reserva.

El importe se obtiene mediante:

$$T = \text{redondear}\Big((P_b + M_s)\, q, 2\Big),$$

donde P_b es el precio base, M_s el modificador de la sesión y q la cantidad de cupos. Por ejemplo, con precio base de 30 unidades monetarias, modificador de 5 y dos cupos, el total calculado es 70.

Validación de voucher. Se consulta la reserva junto con su sesión, tour, operador y voucher. Se comprueba la propiedad y el estado; después se realiza un cambio condicionado de confirmado a completado. Si otra solicitud ya realizó el cambio, se rechaza una segunda validación. El voucher usado y la bitácora se guardan en la misma transacción. El control condicional evita depender únicamente de una lectura previa del estado.

Registro documental. Primero se validan datos y archivos; después se crea usuario, operador y documentos dentro de una transacción. Si falla la persistencia, se intentan retirar los archivos escritos para no conservar documentos sin expediente. La sustitución conserva el archivo anterior hasta confirmar la nueva referencia. El servicio coordina la escritura de archivos con la persistencia del expediente para conservar su integridad.

Selección en cascada. La pantalla de reservas carga tours, luego sesiones y finalmente reservas. Al cambiar el nivel superior se limpian las selecciones dependientes y se suprimen eventos disparados durante la recarga. El objetivo es evitar que una tabla muestre reservas de una salida distinta a la seleccionada.

5.6. Evidencia y funcionamiento

5.6.1. Capturas de pantalla por módulo

La interfaz se organiza alrededor de las tareas del operador y del administrador. Cada pantalla mantiene un propósito definido y presenta los datos de la actividad seleccionada:

Inicio de sesión. Formulario de correo y contraseña, validación de acceso y comunicación de errores.

Operadores. Listado administrativo con filtros por estado, datos comerciales y acceso al expediente.

Documentos. Consulta de RUC, identificación y documento institucional, carga de archivos y reenvío de la solicitud.

Tours. Listado de ofertas y formulario con título, descripción, destino, precio y duración.

Sesiones. Selección de fecha, horario, disponibilidad y modificador de precio de cada salida.

Reservas. Selección de tour y sesión, consulta de reservas y acción de validación del voucher.

Panel administrativo. Indicadores de operadores pendientes y verificados, tours activos y reservas confirmadas.

5.6.2. Flujos de ejemplo

Flujo 1: alta y revisión de una agencia. El operador registra sus datos y los tres documentos requeridos. El sistema crea el expediente con estado pendiente. El administrador consulta los documentos y aprueba la cuenta o registra el motivo de rechazo. Si se requiere una corrección, el operador sustituye los archivos y reenvía la solicitud de verificación.

Flujo 2: publicación y programación. Una agencia aprobada inicia sesión, crea un tour y registra una salida con diez cupos. El catálogo conserva la oferta y la sesión queda disponible para consulta. Si una cuenta pendiente intenta crear un tour desde el servicio del operador, el sistema deniega la operación y comunica que necesita aprobación.

Flujo 3: reserva y cierre. Un usuario autenticado solicita por API dos cupos de una sesión con diez disponibles. La transacción registra la reserva confirmada y actualiza el cupo a ocho. El operador selecciona el tour y su sesión en el backoffice, consulta la reserva y valida la atención del servicio. La reserva pasa a completada y el voucher queda utilizado. Una segunda validación produce un mensaje de reutilización.

Flujo 4: control de disponibilidad. Cuando una sesión conserva un cupo y se solicitan dos, el servicio responde que no hay disponibilidad suficiente. La actualización condicionada impide descontar una cantidad superior al cupo restante. La reserva y el descuento se ejecutan dentro de la misma transacción para conservar la consistencia de la operación.

5.7. Repositorio GitHub

5.7.1. Enlace del repositorio

El repositorio oficial es `https://github.com/AJcano09/explora-nicaragua`. Contiene la solución, los proyectos de dominio y aplicación, los servicios de infraestructura, la API y ambos clientes Windows Forms [**?**]. Git conserva la evolución del código y permite recuperar el estado de cada componente.

La raíz reúne las instrucciones de instalación, la configuración de servicios y los scripts de preparación. El código de la aplicación se concentra en `src`, mientras `odd/tasks` organiza las tareas técnicas. Los archivos de configuración definen las opciones de cada entorno y las credenciales se suministran mediante variables de entorno o secretos de desarrollo.

5.7.2. Organización del repositorio e historial de cambios

La organización por proyectos permite localizar interfaces, entidades, servicios y clientes sin mezclar sus responsabilidades. El historial utiliza mensajes que identifican el componente y el tipo de cambio: funcionalidades, correcciones, refactorizaciones y documentación. La siguiente tabla resume los hitos principales del desarrollo.

Tabla 5.4: Historial de cambios de ExploraNica.

Fecha	Commit	Cambio registrado
10/10/2026	`fe8c449`	Indicadores de tours activos y reservas confirmadas en administración.
11/10/2026	`0b21e52`	Endpoint y acción de escritorio para validar vouchers.
11/10/2026	`9f4d173`	Control de concurrencia en validación de vouchers.
11/10/2026	`96824c0`	Transacción de reserva dentro de la estrategia de ejecución.
12/10/2026	`a63af9c`	Registro público de operadores con documentación.
12/10/2026	`54ea7af`	Actualización de operadores y endpoints documentales.
12/10/2026	`88236ac`	Vistas de detalle, edición y documentos en administración.
12/10/2026	`ea75b35`	Carga y visualización documental para operadores.
12/10/2026	`75eccef`	Centralización del tratamiento de errores asíncronos de interfaz.
12/10/2026	`33ce0d8`	Traducción de mensajes visibles pendientes al español.

El desarrollo se realizó de forma incremental. Primero se incorporaron la gestión de tours y los indicadores administrativos; después se añadieron la validación de vouchers, los controles de concurrencia, el registro documental y las pantallas de verificación. La centralización de errores asíncronos y la traducción de mensajes completaron la integración de los clientes de escritorio.

5.8. Documentación técnica básica

5.8.1. Guía de instalación y ejecución

Requisitos. Los clientes requieren Windows. El entorno de desarrollo utiliza SDK .NET 10, Git y PostgreSQL. Docker Compose permite iniciar PostgreSQL 16 y los servicios auxiliares definidos en el proyecto. La API y los dos clientes se ejecutan como procesos independientes.

Preparación. Para una copia nueva:

```
git clone https://github.com/AJcano09/explora-nicaragua.git
cd explora-nicaragua
dotnet restore ExploraNica.slnx
dotnet build ExploraNica.slnx -c Release
```

Para trabajar con una copia existente, abrir su carpeta raíz. Con Docker Compose, copiar `.env.example` a `.env`, definir las variables de los servicios y ejecutar `dockercomposeup-d explora-db`. Con PostgreSQL instalado directamente, crear la base de datos y configurar la cadena de conexión. La API recibe su configuración mediante `appsettings.json` y variables de entorno; el archivo `.env` pertenece a la configuración de Docker.

Configuración de API. Establecer `ConnectionStrings:DefaultConnection`, `Jwt:Key`, `Jwt:Issuer`, `Jwt:Audience` y `Jwt:ExpiryMinutes`. Usar secretos de desarrollo o variables de entorno con doble guion bajo, por ejemplo `ConnectionStrings__DefaultConnection`. Mantener los secretos fuera del control de versiones. La carga inicial puede crear usuarios de demostración; el hash de sus contraseñas sólo se habilita en desarrollo cuando se suministra `Seed:DevPassword`.

Base de datos. Instalar una versión 10.x de `dotnet-ef` compatible con los paquetes Entity Framework Core 10 del proyecto. Ejecutar desde la raíz:

```
dotnet ef database update --project src/Infrastructure/ExploraNica.Infrastructure
↪  --startup-project src/Presentation/ExploraNica.Api
```

La API también intenta migrar e inicializar datos al arrancar. Si la base de datos falla, puede mantener el proceso vivo aunque no esté listo para atender operaciones. Por eso se deben comprobar los dos endpoints de salud.

Ejecución de API y clientes. En terminales independientes:

```
dotnet run --project src/Presentation/ExploraNica.Api --launch-profile https
dotnet run --project src/Presentation/ExploraNica.OperatorApp
dotnet run --project src/Presentation/ExploraNica.AdminApp
```

El perfil HTTPS declara `https://localhost:7152` y el perfil HTTP `http://localhost:5241`. Configurar `Api:BaseUrl` de ambos clientes con la dirección elegida. En desarrollo puede ser necesario preparar el certificado local con `dotnetdev-certshttps--trust`. Revisar `/health/live`, `/health/ready` y, en desarrollo, `/swagger`. El endpoint de disponibilidad debe responder satisfactoriamente antes de probar operaciones que dependan de PostgreSQL.

Verificación de instalación. Comprobar la compilación de la solución, la aplicación de las migraciones y la disponibilidad de la base de datos. Iniciar la API y ambos clientes, verificar la dirección del servidor e iniciar sesión con el rol correspondiente. Los endpoints de salud permiten distinguir entre proceso activo y servicio listo para atender solicitudes.

5.8.2. Manual de usuario

Acceso. Abrir el cliente correspondiente, escribir correo y contraseña y pulsar iniciar sesión. Si aparece un error de credenciales, corregir los datos; si el problema es de conexión, comprobar la API. No compartir cuentas. Al concluir las tareas, utilizar la opción de cerrar sesión.

Operador: panel y tours. Consultar el estado de la cuenta y el resumen disponible. En "Mis Tours", revisar el listado y abrir la creación o edición. Completar título, descripción, ubicación, precio y duración; guardar y comprobar que el listado refleja la modificación. La creación de ofertas exige aprobación de la agencia en el servicio correspondiente.

Operador: documentos. Abrir la gestión documental desde el acceso disponible en el panel. Consultar los tres tipos requeridos y adjuntar el archivo correcto para cada uno. En caso de rechazo, revisar el motivo, sustituir los documentos permitidos y reenviar la solicitud. Una cuenta aprobada no admite sustitución documental en el flujo actual.

Operador: sesiones y reservas. Seleccionar el tour y registrar la salida mediante el diálogo de sesiones. En "Reservas", seleccionar primero el tour y después su sesión; comprobar fecha, cupos y reserva antes de pulsar la validación. Validar sólo al verificar la atención del servicio, porque esta acción completa la reserva. El registro de reservas se realiza mediante la API.

Administrador. Consultar el panel de control, abrir operadores, filtrar por estado y revisar datos y documentos. Aprobar tras la revisión o rechazar indicando un motivo comprensible. En tours, activar o desactivar la oferta seleccionada y confirmar el cambio. En reservas, consultar los registros globales y su estado.

Incidencias. Anotar operación, hora e identificador de correlación cuando esté disponible. Antes de reintentar, consultar si la operación fue guardada. No adjuntar documentos personales, claves o tokens a los reportes.

Capítulo 6

Resultados del proyecto

6.1. Resultados obtenidos

Se desarrolló una solución de gestión turística organizada en siete proyectos: dominio, aplicación, infraestructura, componentes compartidos, API, cliente del operador y cliente administrativo. Esta organización separa las entidades, los contratos de servicio, la persistencia y las interfaces de usuario. Los dos clientes de escritorio comparten el mismo servidor y la misma base de datos.

Se implementaron autenticación JWT, verificación documental de operadores, gestión de tours y sesiones, creación de reservas, validación de vouchers e indicadores operativos. La persistencia utiliza Entity Framework Core y PostgreSQL. Las migraciones mantienen el esquema de datos, el middleware centraliza las excepciones y los endpoints de salud informan sobre el estado del servidor y la conexión a la base de datos.

El registro de agencias crea expedientes con sus tres tipos documentales y estado de verificación. La gestión del catálogo vincula cada tour con su operador y sus salidas. El proceso de reserva calcula el importe y descuenta disponibilidad dentro de una transacción. La validación del voucher completa la reserva, registra su utilización y genera el evento de auditoría correspondiente.

Se incorporaron dos mecanismos de integridad para las operaciones concurrentes: actualización condicionada del cupo y cambio condicionado de la reserva confirmada a completada. La primera operación evita descontar más plazas de las disponibles; la segunda impide que dos validaciones completen repetidamente la misma reserva. El plan de validación funcional incluye estos recorridos y sus casos de excepción.

6.2. Beneficios del sistema

El beneficio funcional esperado es concentrar información antes dispersa entre mensajes, registros manuales y archivos independientes. La asociación de operadores, tours, sesiones y reservas permite consultar la operación con una estructura uniforme. La revisión documental facilita un proceso administrativo explícito y los estados de reserva permiten identificar qué servicios siguen confirmados y cuáles fueron atendidos.

Desde el punto de vista técnico, los contratos de aplicación y la API común ofrecen una base para reutilizar servicios desde ambos clientes. La centralización de reglas facilita corregirlas en el servidor. Los mensajes de dominio y la correlación de errores favorecen el diagnóstico; la bitácora existente permite seguir determinadas acciones críticas, aunque su cobertura debe ampliarse.

Para las agencias, la herramienta organiza la oferta, las salidas y las reservas en una interfaz de escritorio. Para la administración, los expedientes y los filtros por estado facilitan la distribución de tareas de verificación. La consulta centralizada permite localizar registros sin recorrer conversaciones o archivos independientes.

6.3. Limitaciones del prototipo

Los clientes de escritorio funcionan en Windows y necesitan conectividad con la API para las operaciones remotas. Los documentos se almacenan en el servidor mediante un servicio de archivos local. La continuidad operativa requiere respaldar tanto PostgreSQL como los archivos documentales y conservar su correspondencia.

Los pagos en línea, la conciliación, los reembolsos, las reseñas verificadas, los reportes financieros y la aplicación móvil pertenecen a la ampliación funcional del sistema. La reserva actual se confirma desde la API y no ejecuta un cargo de pasarela. El voucher registra el código y su utilización durante el cierre de la reserva; la emisión anticipada y lectura de una imagen QR requieren un flujo específico.

La autenticación utiliza tokens con vencimiento y el servidor no expone un servicio de renovación. El modelo reúne al personal de la agencia bajo el rol TourOperator. La analítica se concentra en conteos de control; el análisis financiero necesita sus propias entidades y reglas. La cancelación de reservas y la cobertura de permisos de todas las rutas requieren fortalecer sus controles de estado y propiedad.

La evolución del prototipo comprende pruebas automatizadas, evaluación de concurrencia, accesibilidad y recuperación ante fallos. La operación comercial también requiere procedimientos de soporte, despliegue, monitoreo y atención de incidencias.

6.4. Propuestas de mejora

Se propone priorizar cuatro etapas. La primera debe cerrar la coherencia del núcleo: permisos por propiedad en todas las rutas, validación uniforme, cancelación idempotente, renovación segura de sesión y pruebas de disponibilidad y validación concurrentes. La segunda debe completar evidencias: pruebas automatizadas, capturas por módulo y un piloto con datos de demostración y operadores voluntarios.

La tercera etapa puede incorporar pagos en entorno de prueba, estados de pago separados de los de reserva, procesamiento idempotente de eventos, conciliación y manejo de cancelaciones. Los eventos de la pasarela deben validarse y no asumirse únicos ni necesariamente ordenados [**?**]. La selección de proveedor deberá confirmar su disponibilidad comercial y operativa para el proyecto antes de efectuar cobros reales.

La cuarta etapa puede desarrollar reseñas vinculadas a servicios atendidos, reportes de bruto, comisión y neto, interfaz móvil y mejoras de accesibilidad. Cada ampliación necesita criterios de aceptación, responsable y evidencia. El orden prioriza integridad de datos y demostración del núcleo antes de ampliar el catálogo de funciones.

Capítulo 7

Impacto, ética e identidad nacional

7.1. Impacto tecnológico

ExploraNica aplica una arquitectura de servicios centralizados a la gestión operativa del turismo. Los clientes de escritorio comparten una API y una base de datos con contratos, estados y relaciones definidos. Esta organización concentra las operaciones sobre una fuente común y permite ampliar los servicios sin duplicar la persistencia en cada cliente.

El proyecto integra programación orientada a objetos, persistencia relacional, autenticación, consumo de servicios, programación asíncrona y control de versiones. La combinación de estas tecnologías permite desarrollar interfaces de gestión y coordinar operaciones sobre datos compartidos. La separación de componentes facilita mantener el producto y ampliar sus servicios.

La sostenibilidad tecnológica se apoya en dependencias mantenidas, decisiones de arquitectura documentadas y separación entre clientes y servidor. Los contratos permiten evolucionar los componentes de presentación y mantener las reglas de negocio en los servicios de la API.

7.2. Impacto económico

ExploraNica organiza la oferta de las agencias y facilita su incorporación a procesos digitales. La Cuenta Satélite de Turismo del Banco Central de Nicaragua describe la producción, el consumo y el aporte económico del sector y sirve como referencia para analizar su contexto [**?**]. El proyecto se orienta a las necesidades de gestión de los prestadores de servicios turísticos.

El modelo de negocio plantea una comisión del 8 % por reserva. Para una base de 100 unidades monetarias, esta regla corresponde a una comisión de 8 y un saldo preliminar de 92, antes de costos de pasarela, impuestos y devoluciones. Su aplicación requiere incorporar

el módulo financiero, registrar las transacciones y definir las reglas de conciliación.

La evaluación económica futura debe registrar costo de infraestructura, mantenimiento, capacitación y soporte; reservas atendidas; ingresos efectivamente conciliados y tasa de cancelación. Sólo con estos datos podrá valorarse si la herramienta aporta ahorro operativo o rentabilidad suficiente para sostener su servicio.

7.3. Impacto social

La digitalización facilita que las agencias de menor tamaño organicen su oferta con una estructura común de destinos, precios, fechas y responsables. La administración de expedientes y estados establece un procedimiento de revisión compartido. La claridad de la información y la identificación del operador contribuyen a una relación más ordenada entre prestadores y usuarios.

El acceso también puede verse limitado por conectividad, disponibilidad de equipos Windows y experiencia digital. El piloto debe incluir capacitación breve, tareas comprensibles y mecanismos de soporte. La inclusión requiere probar legibilidad, navegación por teclado, contraste y claridad de los mensajes, además de considerar agencias fuera de los centros urbanos.

El sistema complementa la atención humana mediante la organización del servicio y el registro de las decisiones. La atención de incidentes de viaje, disputas y necesidades de accesibilidad permanece a cargo de las personas responsables. La capacitación y el soporte facilitan su incorporación a la operación de las agencias.

7.4. Ética profesional en el desarrollo tecnológico

La ética profesional orienta el diseño de reglas transparentes, la protección de datos y el tratamiento responsable de la información turística. El sistema diferencia las responsabilidades del administrador, el operador y el turista. Las operaciones críticas utilizan controles de identidad y estado para proteger la información y mantener la integridad del servicio.

La protección de datos comprende acceso restringido a los expedientes, uso de información necesaria para prestar el servicio y gestión de su conservación. Las interfaces de consulta limitan los datos documentales expuestos. Las credenciales, los tokens y las claves de firma se administran como secretos de configuración y se excluyen del material de capacitación.

La autorización debe aplicarse en el servidor y comprobar la relación entre usuario y recurso. El equipo debe revisar los accesos por rol y documentar incidentes sin exponer datos personales. La bitácora debe identificar al responsable real, proteger su integridad y evitar almacenar secretos. Los registros actuales con responsable genérico requieren mejorarse para

ofrecer trazabilidad individual.

La verificación de agencias comunica su estado y el motivo de rechazo para permitir la corrección del expediente. La incorporación de reseñas y pagos requiere reglas de moderación, cargos, cancelación y reclamación comprensibles para el usuario. La operación del servicio debe integrar los requisitos normativos aplicables al tratamiento de datos y a la actividad turística.

7.5. Historia e identidad nacional en el proyecto

La identidad nacional se expresa en la selección de Nicaragua como territorio de referencia y en la vinculación de sus destinos con operadores locales. Ometepe, Somoto y San Juan del Sur representan escenarios diversos de paisaje y actividad turística. El catálogo permite organizar experiencias asociadas con destinos y prestadores del país.

El catálogo debe presentar la historia y cultura de cada destino con fuentes identificadas y revisión local. Las descripciones de patrimonio, acontecimientos y tradiciones deben evitar errores, estereotipos o apropiación de expresiones comunitarias. Cuando se incorporen fotografías, relatos o conocimientos de una comunidad, debe verificarse su autorización de uso y reconocerse su procedencia.

La utilización del español en los mensajes y de una identidad visual vinculada al país contribuye a la familiaridad del producto. No obstante, los símbolos y nombres nacionales por sí solos no garantizan pertinencia cultural. Esta debe evaluarse mediante participación de operadores y comunidades, respeto por sus formas de representación y actualización responsable de la información.

La proyección turística debe equilibrar promoción y cuidado del entorno: describir recomendaciones de visita, capacidad de atención y condiciones del servicio, sin prometer acceso irrestricto a espacios sensibles. El sistema puede incorporar campos de procedencia y fecha de revisión del contenido para mantener responsabilidad editorial.

7.6. Proyección

La proyección inmediata es consolidar un prototipo demostrable para agencias y administración, con instalación reproducible, pruebas del núcleo y evidencias de uso. La siguiente etapa es un piloto acotado en el que se midan tareas concretas: crear una oferta, programar una salida, consultar reservas y validar atención. Cada participante debe conocer el propósito de la evaluación y el tratamiento de sus datos.

Si el piloto acredita utilidad, puede avanzarse hacia pagos de prueba, reportes y acceso móvil, sin comprometer de antemano una operación comercial a escala nacional. La expansión

requiere responsables de soporte, presupuesto de mantenimiento, respaldo y recuperación, acuerdos con operadores y políticas de atención al usuario.

La proyección de largo plazo es consolidar una plataforma de gestión turística con participación local y mejora continua. La expansión del producto se apoya en estabilidad técnica, uso sostenido y sostenibilidad económica. Los nuevos módulos deben responder a necesidades concretas de operadores y turistas, con criterios de aceptación definidos.

Capítulo 8

Conclusiones, recomendaciones y anexos

8.1. Conclusiones

El proyecto traduce la necesidad de organizar la oferta turística y la gestión de agencias en un prototipo con clientes de escritorio y servicios compartidos. La relación entre operadores, tours, sesiones y reservas ofrece una base concreta para digitalizar tareas internas descritas en el diagnóstico y los procesos propuestos.

Se implementaron autenticación, verificación documental, catálogo, disponibilidad, reservas y validación de vouchers. La solución utiliza .NET 10, Windows Forms, ASP.NET Core, Entity Framework Core y PostgreSQL. La organización por proyectos y las migraciones permiten mantener los componentes y la evolución del esquema de datos.

La gestión interna constituye el núcleo del sistema. Las integraciones de pago, reseñas, reportes financieros y aplicación móvil conforman su ruta de ampliación. La arquitectura de servicios permite incorporar estos componentes sobre las entidades y procesos existentes, manteniendo las reglas de identidad, propiedad y estado.

Las transacciones de reserva y los cambios condicionados de disponibilidad y estado aportan mecanismos concretos de integridad. La validación del voucher relaciona al operador con la reserva y registra el cierre del servicio. La correlación de errores y los eventos de auditoría facilitan el seguimiento de incidencias y operaciones críticas.

Finalmente, el valor del proyecto depende de combinar desarrollo técnico con responsabilidad sobre los datos, transparencia de las decisiones e información cultural verificable. La pertinencia nacional se fortalecerá mediante participación de operadores y comunidades, documentación completa y evaluación de las necesidades reales del sector.

8.2. Recomendaciones

1. Consolidar una versión de entrega identificada por commit y etiqueta. Separar configuración local de código publicado y documentar el procedimiento de actualización.
2. Revisar permisos y pertenencia de recursos en todos los endpoints; homogeneizar validaciones y cerrar la renovación de sesión y la cancelación repetida de reservas.
3. Incorporar pruebas automatizadas de reserva, disponibilidad y voucher. Incluir fallos de base de datos, reintentos y solicitudes concurrentes, y conservar los resultados.
4. Mantener el manual de usuario y el registro de pruebas alineados con las funcionalidades del sistema. Utilizar datos de demostración anonimizados en la capacitación.
5. Realizar un piloto con tareas y métricas definidas antes de valorar ahorro, satisfacción o aceptación. Recoger dificultades de conectividad y aprendizaje, además de resultados positivos.
6. Incorporar pagos primero en entorno de prueba, con estados separados, validación de eventos, idempotencia y conciliación. Evaluar costos y disponibilidad del proveedor antes de operar.
7. Desarrollar reportes financieros y reseñas después de estabilizar el núcleo, definiendo fuentes, reglas de cálculo, moderación y derechos de reclamación.
8. Fortalecer respaldos de PostgreSQL y archivos documentales, recuperación y trazabilidad del usuario real. Mantener secretos fuera del repositorio y limitar la retención de documentos.
9. Revisar accesibilidad, mensajes y representación cultural con usuarios locales. Registrar procedencia y derechos de uso de fotografías y textos de los destinos.
10. Establecer un ciclo periódico de mantenimiento, actualización de dependencias y revisión de la información de los destinos.

8.3. Referencias bibliográficas

La fundamentación del sistema reúne fuentes de arquitectura de software, desarrollo de escritorio, autenticación, modelado de procesos y gestión turística.

[1] Instituto Nicaragüense de Turismo. *Oferta turística: agencias de viajes*. Mapa Nacional de Turismo. `https://www.mapanicaragua.com/oferta-turistica-agencias-de-viajes/`.

[2] Banco Central de Nicaragua. *Informe de resultados de la Cuenta Satélite de Turismo de Nicaragua en 2023*. https://www.bcn.gob.ni/publicaciones/informe-de-resultados-de-la-cuenta-satelite-de-turismo-de-nicaragua-en-2023.

[3] Ries, E. (2011). *The Lean Startup*. Crown Business.

[4] Osterwalder, A. y Pigneur, Y. (2010). *Business Model Generation*. Wiley.

[5] React Native. *Documentación oficial*. https://reactnative.dev/.

[6] Civitatis. *Proveedores: preguntas frecuentes*. https://www.civitatis.com/es/proveedores/landing-preguntas-frecuentes/.

[7] Hardt, D. (ed.) (2012). *The OAuth 2.0 Authorization Framework*. RFC 6749. https://www.rfc-editor.org/info/rfc6749/.

[8] Tripadvisor (2023). *Review Transparency Report*. https://www.tripadvisor.com/TransparencyReport2023.

[9] Mozilla Developer Network. *Offline and background operation*. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation.

[10] Firebase. *Firebase Cloud Messaging*. https://firebase.google.com/docs/cloud-messaging.

[11] Object Management Group (2011). *Business Process Model and Notation, Version 2.0*. https://www.omg.org/spec/BPMN/2.0/.

[12] Camunda. *Desktop Modeler*. https://docs.camunda.io/docs/components/modeler/desktop-modeler/.

[13] Maurya, A. (2012). *Running Lean*. O'Reilly Media.

[14] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Tesis doctoral, University of California, Irvine. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[15] Microsoft. *Windows Forms overview*. https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/.

[16] Microsoft. *Documentación de .NET*. https://learn.microsoft.com/dotnet/.

[17] NIST. *Role Based Access Control*. https://csrc.nist.gov/projects/role-based-access-control/faqs.

[18] Stripe. *Receive Stripe events in your webhook endpoint.* https://docs.stripe.com/webhooks.

[19] ExploraNica. *Repositorio oficial del sistema.* https://github.com/AJcano09/explora-nicaragua.

[20] Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT).* RFC 7519. https://www.rfc-editor.org/info/rfc7519/.

8.4. Anexos

Anexo A. Matriz de verificación funcional

El plan de validación funcional define las operaciones principales y sus condiciones de aceptación. Cada caso establece una acción, la respuesta del sistema y la comprobación de integridad correspondiente.

Caso	Acción	Condición de aceptación
A01	Iniciar sesión con datos válidos e inválidos.	Acceso por rol o rechazo; captura sin credenciales y respuesta HTTP.
A02	Registrar operador con tres documentos válidos.	Cuenta pendiente y expediente completo; identificador anonimizado.
A03	Registrar correo o RUC repetido.	Conflicto controlado; sin duplicados ni archivos huérfanos.
A04	Adjuntar archivo inválido o mayor a 5 MiB.	Rechazo por campo; sin persistencia parcial.
A05	Aprobar, rechazar y reenviar una solicitud.	Estados y motivo coherentes; capturas antes y después.
A06	Crear un tour con cuenta aprobada y pendiente.	Creación permitida o denegada según ruta y permiso.
A07	Consultar y editar recursos de otra agencia.	Denegación sin exposición de datos.
A08	Reservar dos cupos de una sesión con diez.	Reserva confirmada y disponibilidad igual a ocho.
A09	Disputar concurrentemente el último cupo.	Una confirmación; otra solicitud rechazada; cupo no negativo.

Caso	Acción	Condición de aceptación
A10	Validar dos veces la misma reserva.	Una transición a completada y rechazo de reutilización.
A11	Validar una reserva ajena.	Sin cambio de estado ni acceso a datos de otro operador.
A12	Cancelar repetidamente una reserva.	Probar idempotencia y cupos; registrar cualquier desviación.
A13	Interrumpir base de datos o conectividad.	Error comprensible, correlación y recuperación documentada.
A14	Comprobar vencimiento de sesión.	Solicitud de autenticación al expirar el token y acceso restablecido tras iniciar sesión.

El registro de ejecución incluye fecha, entorno, datos iniciales, pasos, resultado, responsable y evidencia. Las incidencias se relacionan con el caso que las detecta y con el cambio que las resuelve.

Anexo B. Registro de evidencia visual

La documentación visual se organiza por tarea: acceso al sistema, gestión de operadores, expediente documental, catálogo de tours, sesiones, reservas y panel administrativo. Los identificadores internos de estas pantallas son FormLogin, UcVendors, OperatorDocumentsForm, OperatorToursForm, SessionDialog, OperatorBookingsForm y UcDashboard. Las imágenes de soporte se conservan en `assets/evidencias` y se vinculan con el manual de usuario.

Anexo C. Rutas de referencia de la API

Método	Ruta	Propósito
POST	`/api/auth/login`	Inicio de sesión.
POST	`/api/operators`	Registro con documentos.
GET	`/api/operators/me/documents`	Expediente propio.
POST	`/api/operators/me/submit-verification`	Reenvío a revisión.
GET	`/api/admin/operators`	Listado administrativo.
POST	`/api/admin/operators/{id}/approve`	Aprobar operador.

Método	Ruta	Propósito
POST	`/api/admin/operators/{id}/reject`	Rechazar con motivo.
GET/POST	`/api/operator/tours`	Consultar o crear tours propios.
POST	`/api/bookings`	Crear reserva autenticada.
POST	`/api/operator/tours/bookings/{id}/validate`	Validar una reserva del operador.
GET	`/health/live` y `/health/ready`	Salud de proceso y disponibilidad.

OpenAPI publica los contratos de la API y Swagger permite consultar las rutas durante el desarrollo. Las solicitudes de los recursos protegidos incluyen el token de acceso, los identificadores de recurso y los campos definidos por cada DTO.

Anexo D. Glosario técnico básico

API REST. Interfaz de servicios que expone recursos mediante operaciones HTTP.

DTO. Objeto para transferir datos entre componentes sin exponer toda la entidad.

JWT. Token firmado con atributos de identidad y vigencia verificables.

Idempotencia. Propiedad por la que repetir una operación no duplica su efecto de negocio.

Migración. Cambio versionado del esquema de base de datos.

Transacción. Conjunto de escrituras que se confirma o revierte como una unidad.

Voucher. Comprobante de reserva con código y registro de utilización.

Verificación documental. Evaluación administrativa del expediente de una agencia.